



**Università
degli Studi
di Palermo**



Introduzione a SpaCy

CORSO DI NATURAL LANGUAGE PROCESSING (ELABORAZIONE DEL LINGUAGGIO NATURALE)
a.a. 2026r/2027

Prof. Roberto Pirrone



Installazione e importazione dei pacchetti

spaCy — NLP industriale in Python

Installazione

Terminale

```
pip install -U spacy
python -m spacy download en_core_web_sm # inglese
python -m spacy download it_core_news_sm # italiano

# modelli più grandi: _md, _lg (vettori), _trf
# estensione per WordNet (slide 11)
pip install spacy-wordnet
pip install nltk
```

Importazione

Python

```
import spacy
print(spacy.__version__) # es. 3.8.16

from spacy.tokens import Doc, Token, Span
from spacy.language import Language
from spacy.matcher import Matcher
from spacy import displacy
```

Nota

Se pip segnala un conflitto di versione tra spacy-wordnet e nltk, si può installare senza dipendenze:

```
pip install spacy-wordnet --no-deps
```

Moduli principali

Modulo	Contenuto
spacy	load, blank, explain, displacy
spacy.tokens	Doc, Token, Span, DocBin
spacy.language	Language, @Language.component
spacy.matcher	Matcher, PhraseMatcher
spacy.pipeline	EntityRuler, Sentencizer...
spacy.symbols	costanti (ORTH, LEMMA...)
spacy.util	filter_spans, minibatch
spacy.lang.xx	dati specifici per lingua

Esempio minimo

Python

```
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp('spaCy makes NLP easy.')
for t in doc:
    print(t.text, t.pos_, t.dep_)
```

Da sapere

A differenza di NLTK, spaCy è orientato agli oggetti: una sola chiamata `nlp(testo)` esegue tutta la pipeline e restituisce un Doc già annotato.

Caricamento delle risorse linguistiche

spacy.load · pipeline · modelli per lingua

Caricare un modello

Python

```
import spacy
nlp = spacy.load('en_core_web_sm')
print(nlp.pipe_names)
print(nlp.meta['name'], nlp.meta['version'],
      nlp.meta['lang'])

it = spacy.load('it_core_news_sm') # italiano
vuoto = spacy.blank('it')          # solo tokenizer
print(vuoto.pipe_names)
```

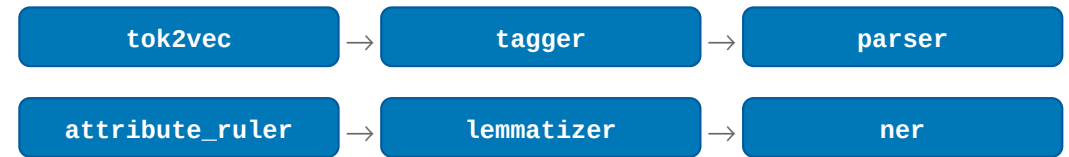
Output

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler',
 'lemmatizer', 'ner']
core_web_sm 3.8.0 en
[]
```

Caricare solo ciò che serve

```
# disable: il componente c'è, ma è spento
nlp = spacy.load('en_core_web_sm',
                 disable=['parser', 'ner'])
# exclude: il componente non viene proprio caricato
nlp = spacy.load('en_core_web_sm', exclude=['parser'])
```

Pipeline di en_core_web_sm



nlp(testo) fa prima la tokenizzazione, poi applica in ordine ogni componente della pipeline al Doc.

Modelli disponibili

Pacchetto	Contenuto
en_core_web_sm	inglese, piccolo, senza vettori
en_core_web_md/lg	inglese con vettori di parole
en_core_web_trf	inglese, basato su transformer
it_core_news_sm	italiano (pipeline diversa*)

* it_core_news_sm ha anche il 'morphologizer' e un ordine diverso dei componenti: controlla sempre nlp.pipe_names del modello che usi.

Tokenizzazione: Doc, Token e Span

il risultato di `nlp(testo)`

Tokenizzazione e frasi

Python

```
doc = nlp("Apple is looking at buying a U.K. startup  
for $1 billion. Don't panic!")  
print([t.text for t in doc])  
print([s.text for s in doc.sents])  
print(len(doc), doc[0], doc[0:3])
```

Output

```
['Apple', 'is', 'looking', 'at', 'buying', 'a', 'U.K.',  
 'startup', 'for', '$', '1', 'billion', '.', 'Do', 'n't',  
 'panic', '!']  
['Apple is looking at buying a U.K. startup for  
$1 billion.', "Don't panic!"]  
17 Apple Apple is looking
```

Da sapere

Il tokenizzatore usa regole e eccezioni, non solo gli spazi:
'U.K.' resta unito, "Don't" diventa 'Do' + "n't", '\$' è
separato da '1'. Il testo originale resta in `doc.text`.

Gli oggetti fondamentali

Oggetto	Cos'è
Doc	sequenza di token con le annotazioni
Token	singola unità (<code>doc[i]</code>)
Span	porzione di Doc (<code>doc[i:j]</code>)
Vocab	lessico condiviso (<code>nlp.vocab</code>)

Attributi utili di un Token

Python

```
d = nlp('The runners were running quickly')  
t = d[1]  
print(t.text, t.i, t.idx, t.lower_, t.shape_)  
print(t.is_alpha, t.is_stop, t.like_num)  
# .text .i (posizione) .idx (offset carattere)  
# .is_punct .is_title .like_url .like_email
```

Output

```
runners 1 4 runners xxxx  
True False False
```

Tipologie di tokenizzazione

regole, casi speciali, spiegazione, segmentazione in frasi

Come ragiona il tokenizzatore

Python

```
print(nlp.tokenizer.explain("Let's go to N.Y.!"))
t = nlp.make_doc("Let's go to N.Y.! mail a@b.it")
print([x.text for x in t])
```

Output

```
[('SPECIAL-1', 'Let'), ('SPECIAL-2', "'s"), ('TOKEN', 'go'),
 ('TOKEN', 'to'), ('SPECIAL-1', 'N.Y.'), ('SUFFIX', '!')]
['Let', "'s", 'go', 'to', 'N.Y.', '!', 'mail',
 'a@b.it']
```

Caso speciale personalizzato

Python

```
from spacy.symbols import ORTH
nlp.tokenizer.add_special_case('gonna',
    [{ORTH: 'gon'}, {ORTH: 'na'}])
print([x.text for x in nlp.make_doc("I'm gonna go")])
```

Output

```
['I', "'m", 'gon', 'na', 'go']
```

Italiano: elisioni e apostrofi

Python

```
bl = spacy.blank('it')
doc = bl("L'università di Palermo è bella,
        dell'ingegneria.")
print([t.text for t in doc])
```

Output

```
["L'", 'università', 'di', 'Palermo', 'è', 'bella', ',', ' ',
 "dell'", 'ingegneria', '.']
```

Segmentazione in frasi

Python

```
# 1) parser (default nei modelli completi)
# 2) sentencizer: solo regole sulla punteggiatura
b = spacy.blank('en'); b.add_pipe('sentencizer')
txt = 'Hello there. How are you? Fine!'
print([s.text for s in b(txt).sents])
```

Output

```
['Hello there.', 'How are you?', 'Fine!']
```

Il tokenizzatore di spaCy è basato su regole (parole);
i modelli `_trf` usano internamente anche i subword.

Stemming

spaCy non include stemmer: si usa NLTK come estensione

Stemmer Snowball come attributo del token

Python

```
from nltk.stem import SnowballStemmer
from spacy.tokens import Token

stemmer = SnowballStemmer('english')
Token.set_extension('stem',
    getter=lambda t: stemmer.stem(t.text.lower()))

d = nlp('The runners were running quickly and
    studies studied')
print([(t.text, t._.stem) for t in d])
```

Output

```
[('The', 'the'), ('runners', 'runner'), ('were', 'were'),
 ('running', 'run'), ('quickly', 'quick'), ('and', 'and'),
 ('studies', 'studi'), ('studied', 'studi')]
```

Perché serve un'estensione

spaCy offre la lemmatizzazione, non lo stemming: gli stemmer (Porter, Snowball) sono in NLTK. L'estensione 'stem' li rende disponibili come `t._.stem` su ogni token.

Stemming o lemmatizzazione?

Parola	Stem	Lemma
studies	studi	study
quickly	quick	quickly
were	were	be
running	run	run

Attenzione

Lo stem può non essere una parola vera ('studi'); il lemma è una forma del dizionario ('study') e sfrutta il POS. Lo stemming è più veloce ma meno preciso.

Per l'italiano: `SnowballStemmer('italian')`.

Lemmatizzazione

token.lemma_ · pipeline lemmatizer

Inglese

Python

```
d = nlp('The mice were running and the
        children studied')
print([(t.text, t.lemma_, t.pos_) for t in d])
print(nlp.get_pipe('lemmatizer').mode)
```

Output

```
[('The', 'the', 'DET'), ('mice', 'mouse', 'NOUN'),
 ('were', 'be', 'AUX'), ('running', 'run', 'VERB'),
 ('and', 'and', 'CCONJ'), ('the', 'the', 'DET'),
 ('children', 'child', 'NOUN'), ('studied', 'study', 'VERB')]
rule
```

Da sapere

Il lemmatizer inglese è basato su regole + tabelle e usa il POS assegnato prima (attribute_ruler): senza tagger i lemmi sono meno affidabili.

Italiano

Python

```
it = spacy.load('it_core_news_sm')
d = it('Maria ha comprato una macchina a Palermo.')
print([(t.text, t.lemma_) for t in d])
```

Output

```
[('Maria', 'Maria'), ('ha', 'avere'),
 ('comprato', 'comprare'), ('una', 'uno'),
 ('macchina', 'macchina'), ('a', 'a'),
 ('Palermo', 'Palermo'), ('.', '.')]

```

Limiti verificati

I modelli piccoli sbagliano: con 'studiano' e 'studiavano' it_core_news_sm restituisce 'studiano' e 'studiavare' (lemmi corretti: 'studiare'). Per più precisione si usano modelli più grandi o correzioni con regole proprie.

POS tagging

token.pos_ (universale) · token.tag_ (fine) · token.morph

Etichette grammaticali e dipendenze

Python

```
d = nlp('Apple is buying a small startup in London.')
for t in d:
    print(t.text, t.pos_, t.tag_, t.dep_, t.head.text)
print(spacy.explain('PROPN'))
print(d[1].morph)
```

Output

```
Apple    PROPN NNP nsubj    buying
is       AUX   VBZ aux      buying
buying   VERB   VBG ROOT    buying
a        DET   DT  det      startup
small    ADJ    JJ  amod     startup
startup  NOUN   NN  dobj     buying
in       ADP    IN  prep     buying
London   PROPN NNP pobj     in
proper noun
Mood=Ind|Number=Sing|Person=3|Tense=Pres|VerbForm=Fin
```

Quale etichetta usare

Attributo	Significato
pos_	POS universale (UPOS)
tag_	tag fine del treebank (Penn)
morph	tratti morfologici
dep_	relazione di dipendenza
head	token da cui dipende

Selezionare per categoria

Python

```
nomi = [t.lemma_ for t in d if t.pos_ == 'NOUN']
nomi_propri = [t.text for t in d
                if t.pos_ == 'PROPN']
spacy.explain('nsubj') # 'nominal subject'
```

spacy.explain(etichetta) dà la descrizione di POS, tag, dipendenze ed entità. I tag fini cambiano per lingua.

Named Entity Recognition

doc.ents · ent_type_ · ent_iob_

Entità nel Doc

Python

```
d = nlp('Apple bought a startup in London for
        $1 billion on Monday.')
print([(e.text, e.label_, e.start_char, e.end_char)
       for e in d.ents])
print([(t.text, t.ent_type_, t.ent_iob_)
       for t in d[:2]])
print(spacy.explain('GPE'))
```

Output

```
[('Apple', 'ORG', 0, 5), ('London', 'GPE', 26, 32),
 ('$1 billion', 'MONEY', 37, 47), ('Monday', 'DATE', 51, 57)]
[('Apple', 'ORG', 'B'), ('bought', '', 'O')]
Countries, cities, states
```

Da sapere

ent_iob_: B = inizio entità, I = dentro, O = fuori.
Le etichette disponibili: nlp.get_pipe('ner').labels

Etichette principali (modello inglese)

Etichetta	Significato
PERSON	persone
ORG	organizzazioni, aziende
GPE	paesi, città, stati
LOC	luoghi non politici
DATE	date e periodi
MONEY	valori monetari
CARDINAL	numeri non classificati altrove

Italiano

Python

```
d = it('Maria ha comprato una macchina a Palermo.')
print([(e.text, e.label_) for e in d.ents])
```

Output

```
[('Maria', 'PER'), ('Palermo', 'LOC')]
```

Attenzione

Le etichette cambiano da modello a modello (PER/LOC in italiano, PERSON/GPE in inglese). Errori possibili:
in 'Gli studenti studiano all'Università di Palermo'
il modello etichetta tutta 'Università di Palermo' come LOC.

N-grammi

finestre di token e noun chunks

N-grammi di token

Python

```
def ngrams(doc, n):
    toks = [t.text.lower() for t in doc if not t.is_punct]
    return [tuple(toks[i:i+n])
            for i in range(len(toks) - n + 1)]

d = nlp('The quick brown fox jumps over the lazy dog.')
print(ngrams(d, 2)[:4])
print(ngrams(d, 3)[:2])
```

Output

```
[('the', 'quick'), ('quick', 'brown'), ('brown', 'fox'),
 ('fox', 'jumps')]
[('the', 'quick', 'brown'), ('quick', 'brown', 'fox')]
```

Frequenze con Counter

```
from collections import Counter
d = nlp('the cat sat on the mat and the cat slept on the mat')
print(Counter(ngrams(d, 2)).most_common(3))
```

Output

```
[(('the', 'cat'), 2), (('on', 'the'), 2), (('the', 'mat'), 2)]
```

N-grammi senza stopwords

Python

```
def ngrams_ok(doc, n):
    out = []
    for i in range(len(doc) - n + 1):
        sp = doc[i:i+n]
        if not any(t.is_stop or t.is_punct for t in sp):
            out.append(sp.text)
    return out

d2 = nlp('The model learns the meaning of natural
language from large text corpora.')
print(ngrams_ok(d2, 2))
```

Output

```
['model learns', 'natural language', 'large text',
 'text corpora']
```

Noun chunks

```
print([n.text for n in d2.noun_chunks])
```

Output

```
['The model', 'the meaning', 'natural language',
 'large text corpora']
```

WordNet con spaCy

estensione spacy-wordnet · synset, lemmi, domini

Aggiungere il componente

Python

```
# serve il corpus: nltk.download('wordnet')
import spacy
from spacy_wordnet.wordnet_annotator import WordnetAnnotator
nlp = spacy.load('en_core_web_sm')
nlp.add_pipe('spacy_wordnet', after='lemmatizer')
print(nlp.pipe_names)
```

Output

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler',
 'lemmatizer', 'spacy_wordnet', 'ner']
```

Synset di un token

Python

```
d = nlp('I deposited money at the bank')
t = d[5]
for s in t._.wordnet.synsets()[3:]:
    print(s.name(), s.definition()[:20])
print(t._.wordnet.lemmas()[3])
```

Output

```
bank.n.01 sloping land (especi
depository_financial_institution.n.01 a financial institut
bank.n.03 a long ridge or pile
[Lemma('bank.n.01.bank'), ...]
```

Navigare in WordNet

Python

```
d = nlp('The dog barked at the cat')
for t in d:
    if t.pos_ in ('NOUN', 'VERB'):
        s = t._.wordnet.synsets()[0]
        print(t.text, s.name(), s.hypernoms())
# Oggetti NLTK: definition(), lemmas(), hypernoms()
```

Output

```
dog   dog.n.01   [canine.n.02, domestic_animal.n.01]
barked bark.v.01 [talk.v.02]
cat   cat.n.01   [feline.n.01]
```

Disambiguazione (Lesk) e domini

Python

```
from nltk.wsd import lesk
b = nlp('I deposited money at the bank')
print(lesk([x.text for x in b], 'bank', 'n'))
print(b[5]._.wordnet.wordnet_domains()[5])
```

Output

```
Synset('savings_bank.n.02')
['geology', 'skiing', 'geography', 'diplomacy', 'book_keeping']
```

Lesk non sempre sceglie il senso giusto e i domini sono rumorosi: valgono come indicazione.

FrameNet con spaCy

frame semantici e ruoli · accesso tramite NLTK

Consultare FrameNet (NLTK)

Python

```
# nltk.download('framenet_v17')
from nltk.corpus import framenet as fn

fs = fn.frames_by_lemma(r'(?i)^buy\.v$')
print([f.name for f in fs])
f = fn.frame('Commerce_buy')
print(sorted(f.FE.keys())[:4])
print(f.FE['Buyer'].coreType)
```

Output atteso (indicativo, non eseguito)

```
['Commerce_buy']
['Buyer', 'Goods', ...]
Core
```

Ruoli dalle dipendenze

```
def roles(doc):
    out = {}
    for t in doc:
        if 'Commerce_buy' in t._.frames:
            for ch in t.children:
                if ch.dep_ == 'nsubj': out['Buyer'] = ch.text
                elif ch.dep_ == 'dobj':
                    out['Goods'] = ' '.join(x.text for x in ch.subtree)
                elif ch.dep_ == 'prep' and ch.lower_ == 'from':
                    out['Seller'] = next(ch.children).text
    return out
```

Componente spaCy: parole → frame

Python

```
from spacy.language import Language
from spacy.tokens import Token
Token.set_extension('frames', default=[])
@Language.component('framenet_tagger')
def framenet_tagger(doc):
    for t in doc:
        if t.pos_ in ('VERB', 'NOUN'):
            p = 'v' if t.pos_ == 'VERB' else 'n'
            fr = fn.frames_by_lemma(
                rf'(?i)^(?{t.lemma_})\.{p}$')
            t._.frames = sorted({f.name for f in fr})
    return doc
nlp.add_pipe('framenet_tagger', after='lemmatizer')
```

Output (frame simulati per l'esempio)

```
d = nlp('Maria bought a car from Luca.')
print([(t.text, t._.frames) for t in d if t._.frames])
[('bought', ['Commerce_buy'])]
print(roles(d))
{'Buyer': 'Maria', 'Goods': 'a car', 'Seller': 'Luca'}
```

Concetti

Un frame è una scena (es. Commerce_buy) con i suoi ruoli: Buyer, Goods, Seller (frame elements, FE). Ogni lexical unit ('buy.v') evoca uno o più frame.

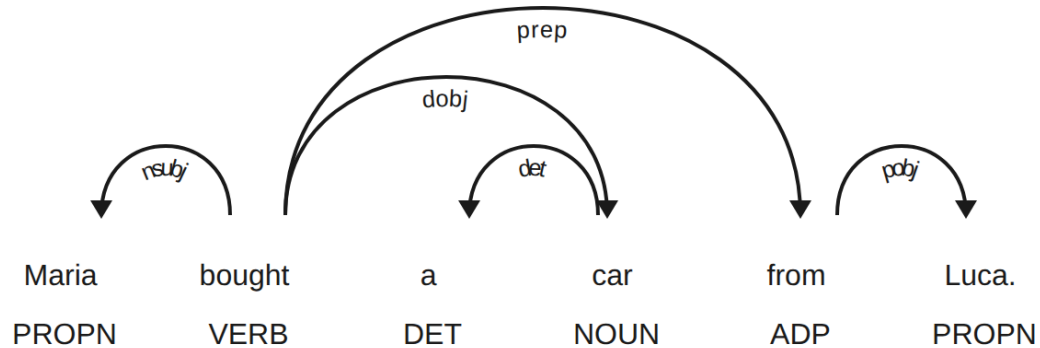
Visualizzazione dei dati: displaCy

alberi di dipendenza, entità e span

Alberi di dipendenza

Python

```
from spacy import displacy
d = nlp('Maria bought a car from Luca.')
displacy.render(d, style='dep', jupyter=True)
# fuori da Jupyter: server locale su localhost:5000
# displacy.serve(d, style='dep')
# oppure SVG/HTML come stringa:
svg = displacy.render(d, style='dep')
open('dep.svg', 'w').write(svg)
```



Entità con evidenziazione

Python

```
d = nlp('Barack Obama was born in Hawaii in 1961
        and worked at Harvard University.')
displacy.render(d, style='ent', jupyter=True)
# colori personalizzati
opts = {'colors': {'ORG': '#F6DCEB'}}
displacy.render(d, style='ent', options=opts)
```

Barack Obama PERSON was born in Hawaii GPE in 1961 DATE and worked at Harvard University ORG .

Da sapere

style='dep' (dipendenze), 'ent' (entità), 'span' (span).
options: colori, distanza, font, compact=True...

Visualizzazione dei dati: span e grafici

displaCy 'span' · matplotlib sulle annotazioni

Span personalizzati

Python

```
d = nlp('Natural language processing with deep learning  
is transforming modern healthcare.')  
d.spans['sc'] = list(d.noun_chunks)  
displacy.render(d, style='span',  
options={'spans_key': 'sc'})
```

Natural language processing with **deep learning** is transforming **modern healthcare** .

NP NP NP

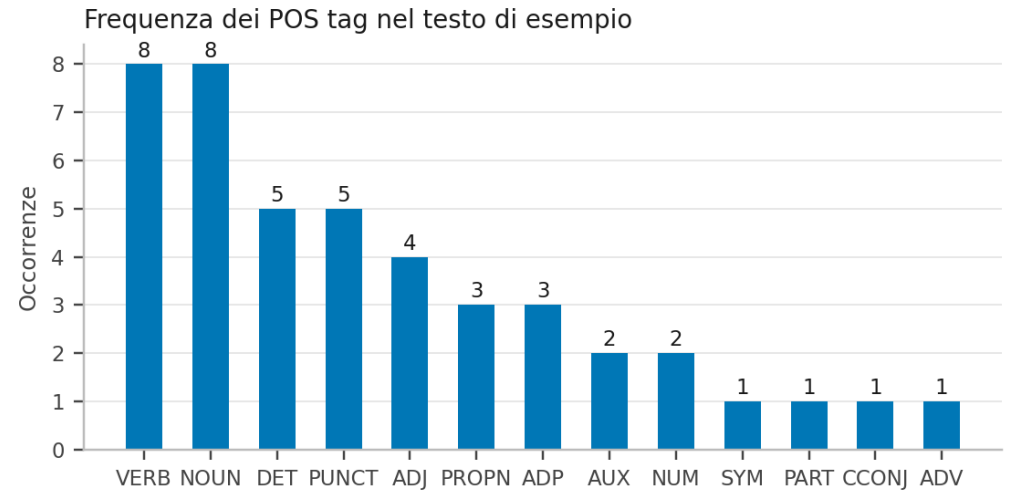
Da sapere

doc.spans è un dizionario di liste di Span (anche sovrapposti): chiave a piacere, qui 'sc'.

Frequenza dei POS con matplotlib

Python

```
from collections import Counter  
import matplotlib.pyplot as plt  
doc = nlp(testo) # testo di esempio di 3 frasi  
cnt = Counter(t.pos_ for t in doc).most_common()  
labels = [k for k, _ in cnt]  
vals = [v for _, v in cnt]  
plt.bar(labels, vals, color='#0077B6')  
plt.ylabel('Occorrenze'); plt.show()
```



Creazione di componenti custom (1)

@Language.component · estensioni · pipeline

Estensioni di Doc e Token

Python

```
from spacy.tokens import Doc, Token
Doc.set_extension('n_content', getter=lambda d:
    sum(1 for t in d if not (t.is_stop or t.is_punct)))
Token.set_extension('is_dose_unit', default=False)
```

Componente-funzione

Python

```
from spacy.language import Language

@Language.component('dose_flagger')
def dose_flagger(doc):
    for t in doc:
        if (t.lower_ in {'mg', 'ml', 'g'} and t.i > 0
            and doc[t.i - 1].like_num):
            t._is_dose_unit = True
    return doc

nlp.add_pipe('dose_flagger', after='tagger')
```

Uso e verifica

Python

```
print(nlp.pipe_names)
d = nlp('The patient takes 500 mg of aspirin twice daily.')
print([t.text for t in d if t._is_dose_unit],
      d._n_content)
```

Output

```
['tok2vec', 'tagger', 'dose_flagger', 'parser',
 'attribute_ruler', 'lemmatizer', 'ner']
['mg'] 7
```

Concetti

Un componente è una funzione che riceve un Doc e lo restituisce. Va registrato con @Language.component e inserito con add_pipe (first, last, before, after).

Verificare e salvare

```
nlp.analyze_pipes(pretty=True) # assegna/richiede
nlp.to_disk('mio_modello')
nlp2 = spacy.load('mio_modello')
```

Creazione di componenti custom (2)

@Language.factory · stato e configurazione · nlp.pipe

Componente con stato e parametri

Python

```
from spacy.matcher import Matcher
from spacy.tokens import Span
from spacy.util import filter_spans

@Language.factory('dose_matcher',
    default_config={'label': 'DOSE',
                    'units': ['mg', 'ml', 'g']})
class DoseMatcher:
    def __init__(self, nlp, name, label, units):
        self.label = label
        self.matcher = Matcher(nlp.vocab)
        self.matcher.add(label, [[{'LIKE_NUM': True},
                                   {'LOWER': {'IN': units}}]])
    def __call__(self, doc):
        spans = [Span(doc, s, e, label=self.label)
                  for _, s, e in self.matcher(doc)]
        doc.ents = filter_spans(spans + list(doc.ents))
        return doc
```

Aggiunta alla pipeline e prova

Python

```
nlp.add_pipe('dose_matcher', after='ner',
             config={'units': ['mg', 'ml']})
d = nlp('The patient takes 500 mg of aspirin and
        5 ml of syrup in Palermo.')
print([(e.text, e.label_) for e in d.ents])
```

Output

```
[('500 mg', 'DOSE'), ('5 ml', 'DOSE'), ('Palermo', 'GPE')]
```

Elaborazione efficiente

Python

```
texts = ['Take 10 mg now.', 'Drink 20 ml daily.']
for d in nlp.pipe(texts, batch_size=2):
    print([e.text for e in d.ents])
with nlp.select_pipes(disable=['parser', 'ner']):
    ... # pipeline ridotta, solo in questo blocco
```

Output

```
['10 mg']
['20 ml', 'daily']
```

Link alle risorse su Internet

documentazione, modelli e strumenti

spaCy

Risorsa	Indirizzo
Sito e documentazione	spacy.io
Guida all'uso (usage)	spacy.io/usage
Riferimento API	spacy.io/api
Modelli e pipeline	spacy.io/models
Corso gratuito	course.spacy.io
Codice sorgente	github.com/explosion/spaCy
Modelli (release)	github.com/explosion/spacy-models
Demo displaCy	demos.explosion.ai

Estensioni usate in queste slide

Risorsa	Indirizzo
spacy-wordnet	pypi.org/project/spacy-wordnet
NLTK	nltk.org
FrameNet	framenet.icsi.berkeley.edu
WordNet	wordnet.princeton.edu

Comandi utili

Terminale

```
python -m spacy info           # versione e modelli
python -m spacy validate       # compatibilità
python -m spacy download <modello>
```

Riepilogo dei concetti

Argomento	API chiave
Caricamento	<code>spacy.load</code> , <code>nlp.pipe_names</code>
Tokenizzazione	<code>nlp()</code> , <code>tokenizer</code> , <code>sentencizer</code>
Lemmi / POS	<code>t.lemma_</code> , <code>t.pos_</code> , <code>t.tag_</code>
Entità	<code>doc.ents</code> , <code>e.label_</code>
N-grammi	<code>slicing doc[i:j]</code> , <code>noun_chunks</code>
Visualizzazione	<code>displacy.render</code>
Componenti	<code>@Language.component/factory</code>

Da sapere

Ogni modello è compatibile con una serie di versioni di spaCy: dopo un aggiornamento di spaCy ricarica i modelli e controlla con 'python -m spacy validate'.